

KI Netzwerk Schweinfurt

Offizielle **statische Website** des KI Netzwerks Schweinfurt — Vernetzung von Unternehmen, Bildung, Verwaltung und Community rund um praxisnahe KI in der Region Mainfranken.

Die Inhalte (Events, Blogbeiträge, Seitentexte, UI-Strings) liegen in **JSON-Dateien** unter `data/`; zusätzlich steuert `data/global/ai-chat.json` den lokalen **WebLLM**-Assistenten (Modelle, Texte, CDN-URLs). Längere Detailtexte liegen in **Markdown** (Event-Detail, Blog-Artikel). Die Pflege erfolgt durch Bearbeiten dieser Dateien — **kein Build-Tool, keine Datenbank, kein Anwendungs-Server** (WebLLM läuft rein im Browser des Besuchers).

Version	0.1.0 (Beta)
Entwicklung	seit April 2026
Entwickler	Oliver Braun — github.com/olli-br
Initiative	Morteza Djebeli Sinaki (Initiator)
Kontakt	ki.netzwerk.schweinfurt@gmail.com

Inhalt

- 1. [Schnellstart](#)
 - 2. [Technik & Architektur](#)
 - 3. [Projektstruktur](#)
 - 4. [Datenstruktur \(data/\)](#)
 - 5. [Metadaten in Dateien \(_meta & Header-Kommentare\)](#)
 - 6. [Neues Event anlegen](#)
 - 7. [Neuen Blogbeitrag anlegen](#)
 - 8. [Markdown-Detailseiten](#)
 - 9. [Seiten- & UI-Texte](#)
 - 10. [Schema: Event](#)
 - 11. [Schema: Blogbeitrag](#)
 - 12. [Schema: About-Seite \(Initiatoren & Partner\)](#)
 - 13. [Schema: Home-Seite \(Sponsoren\)](#)
 - 14. [Bilder & Medien](#)
 - 15. [Lokaler KI-Assistent \(WebLLM\)](#)
 - 16. [Konventionen](#)
 - 17. [Deployment](#)
-

Schnellstart

Zum lokalen Testen im **Projektroot** einen einfachen HTTP-Server starten, damit `fetch` für HTML-Partials, JSON und Markdown zuverlässig funktioniert:

```
cd /pfad/zu/KI-Netzwerk-Schweinfurt-main
python3 -m http.server 8000
```

Im Browser öffnen: <http://localhost:8000> — Einstieg über [index.html](#).

Hinweis: Wenn der HTTP-Server fehlt und die Seite per `file://` geöffnet wird, blockieren die meisten Browser `fetch()`-Aufrufe für lokale Ressourcen. Die Seite zeigt dann automatisch einen Hinweis-Banner mit dem nötigen Befehl.

Technik & Architektur

- **Vanilla HTML / CSS / JavaScript** — keine Frameworks, kein Bundler, kein npm-Build.
- **Mehrsprachig (DE / EN)** — alle sichtbaren Texte als `{ "de": ..., "en": ... }` in JSON; Sprachumschalter in Header und Drawer-Menü.
- **Theme-Toggle** — Hell-/Dunkelmodus, persistiert über `localStorage` (Schlüssel `kins-theme`); Sprache unter `kins-language`.
- **HTML-Partials** — `components/header.html`, `components/footer.html` werden zur Laufzeit per `fetch` in jede Seite eingehängt.
- **Karten-Templates** — `components/event-card.html` und `components/blog-card.html` enthalten Platzhalter wie `{{title}}`, `{{image}}`, `{{detailUrl}}`, die im JS gefüllt werden.
- **Markdown** — längere Inhalte (Event-Detail, Blog-Artikel) liegen in `.md`-Dateien und werden über die `marked`-Bibliothek (CDN-Import on-demand) gerendert; ein leichter Fallback-Renderer ist eingebaut.
- **Karten-Einbettung** — Events mit `geo.lat` / `geo.lng` zeigen eine OpenStreetMap-/Google-Maps-Vorschau plus Direktlinks für Google Maps und Apple Maps.
- **Animationen** — dezentes „Netzwerk“-Canvas in Header und Hero (`<canvas id="header-network-canvas">`, `<canvas id="network-canvas">`); Scroll-Reveal über `.fade-in`.
- **FAQ-Akkordeon** — barrierefrei (`aria-expanded`, Tastaturbedienung).
- **Lokaler KI-Chat (WebLLM)** — schwebender Assistent (`components/ai-chat.html`), lädt `@mlc-ai/web-llm` zur Laufzeit per ES-Module (CDN-URLs konfigurierbar in `data/global/ai-chat.json`). Inferenz erfolgt über **WebGPU** im Browser; Chat-Inhalte verlassen die Seite nicht. Siehe Abschnitt [Lokaler KI-Assistent \(WebLLM\)](#).

Lade-Flow (vereinfacht, in `assets/js/main.js`):

1. Header-, Footer- und AI-Chat-Partial laden, Theme & Sprache initialisieren.
2. Globale UI-Strings (`data/global/global.json`), **KI-Chat-Konfiguration** (`data/global/ai-chat.json`) und Seiten-JSON (`home/`, `events/`, `blog/`, `about/`) laden.
3. Event- & Blog-Listen aus `data/event/index.json` bzw. `data/blog/index.json` lesen, anschließend die einzelnen Einträge nachladen.
4. Markdown-Inhalte werden **on-demand** beim Öffnen der Detailseite geladen.
5. Auf jeder Seite: KI-Assistent initialisieren (Einstellungen aus `localStorage`, optional WebLLM-Engine beim ersten Prompt).

Projektstruktur

```

KI-Netzwerk-Schweinfurt-main/
├── index.html          # Startseite (Hero, nächste Events, Recaps,
Sponsoren, Mission, FAQ, CTA)
├── events.html         # Event-Liste & Event-Detail
├── blog.html           # Blog-Liste & Beitrags-Detail
├── about.html          # Über uns (Initiatoren, Treffen,
Zielgruppe, Partner)
├── README.md
├── README.pdf          # PDF-Export der README
├── assets/
│   ├── css/
│   │   └── styles.css  # Designsystem, Layout, Light-/Dark-Theme,
Cards, Hero, Karten
│   ├── js/
│   │   └── main.js      # App-Logik: Daten, i18n, Theme, Cards,
Markdown, Maps, Countdown, WebLLM-Chat
│   └── images/         # Fotos & Logos
│       ├── meeting1.png
│       ├── meeting2.png
│       ├── event-2.png
│       ├── aboutme_1200x900.png
│       └── StudyFAB_Logo.jpeg
├── components/
│   ├── header.html     # Site-Header mit Navigation, Sprach- &
Theme-Toggle, Burger-Menü
│   ├── footer.html     # Site-Footer (Copyright, Links, Kontakt)
│   ├── event-card.html  # Template für Event-Karten (Listen, Home)
│   ├── blog-card.html   # Template für Blog-Karten (Listen, Home)
│   └── ai-chat.html     # UI des lokalen WebLLM-Assistenten (Toggle,
Einstellungen, Verlauf)
├── data/
│   ├── global/
│   │   ├── global.json # Globale UI-Strings (Nav,
Footer, FAQ, Theme, Common, Countdown)
│   │   └── ai-chat.json # WebLLM: Modelle, CDN-URLs,
Defaults, Chat-Labels (DE/EN)
│   ├── home/
│   │   └── home-page.json # Startseiten-Inhalte (Hero,
Mission, Sponsoren, FAQ-Labels, CTA)
│   ├── about/
│   │   └── about-page.json # Über-uns-Texte, Chips,
Partner, Initiatoren
│   └── event/
│       ├── index.json   # Liste der aktiven Event-
Dateien (`files: [...]`)
│       └── events-page.json # UI-Texte der Events-Seite
(Filter, Detail-Labels, Maps)

```



Hinweis zu zusätzlichen Beispieldateien: Im Verzeichnis `data/event/events/` (sowie `data/blog/posts/`) können weitere `*.json` / `*.md` Beispieldateien liegen, die *aktuell nicht* in `index.json` referenziert werden. Sie sind bewusst entkoppelt — nur Dateien aus dem `files`-Array werden geladen. Solche Beispiele können als Inspiration genutzt oder gelöscht werden.

Datenstruktur (`data/`)

Bereich	Pfad	Zweck
Globale UI-Strings	<code>data/global/global.json</code>	Navigation, Footer, FAQ-Texte, Theme-Labels, Common (Buttons, Loading...), Countdown
KI-Chat / WebLLM	<code>data/global/ai-chat.json</code>	Modell-Katalog (<code>models</code>), ES-Import-URLs (<code>webLlmModuleUrls</code>), Standardeinstellungen (<code>defaultSettings</code>), alle Chat-Strings DE/EN (<code>labels</code>)
Startseite	<code>data/home/home-page.json</code>	Hero, Sektions-Eyebrows, Mission-Features, Sponsoren-Liste, CTA

Bereich	Pfad	Zweck
Über uns	<code>data/about/about-page.json</code>	Hero, Treffenbeschreibung, Teilnehmer- & Partner-Chips, Partner-Logos, Initiatoren mit Bio
Event-Liste	<code>data/event/index.json</code>	Reihenfolge & Auswahl der angezeigten Events (<code>files: [...]</code>)
Events-UI	<code>data/event/events-page.json</code>	Titel, Filter, Detail-Labels, Maps-Beschriftungen
Event-Eintrag	<code>data/event/events/<name>.json</code>	Einzelne Veranstaltung
Event-Detail (lang)	<code>data/event/events/markdown/<name>.{de,en}.md</code>	Ausführlicher Text der Detailseite
Blog-Liste	<code>data/blog/index.json</code>	Reihenfolge & Auswahl der Beiträge (<code>files: [...]</code>)
Blog-UI	<code>data/blog/blog-page.json</code>	Titel, „Weiterlesen“, Lesezeit-Suffix, Recap-Badge, Detail-Zurück
Blog-Eintrag	<code>data/blog/posts/<name>.json</code>	Einzelner Artikel (Metadaten + Teaser)
Blog-Detail (lang)	<code>data/blog/posts/markdown/<name>.{de,en}.md</code>	Volltext des Beitrags
Vorlagen	<code>data/demo-files/*.empty.json</code>	Leere Schemas zum Kopieren

Metadaten in Dateien (`_meta` & Header-Kommentare)

Zur **Nachverfolgung und Pflege** tragen alle relevanten Quell- und Datendateien einheitliche Metadaten:

Art	Inhalt (oben in der Datei)
HTML (<code>*.html</code> , <code>components/*.html</code>)	HTML-Kommentar: <code>Project</code> , <code>Seite</code> , <code>Projekt-Pfad</code> , <code>Description</code> , <code>Developed by</code> , <code>GitHub</code> , <code>Development Start</code> , <code>Current Version</code> , <code>Update History</code>
CSS / JS	Blockkommentar mit denselben Feldern wie HTML
JSON unter <code>data/</code>	Root-Objekt <code>_meta</code> mit den Feldern: <code>project</code> , <code>pageName</code> , <code>pathInProject</code> , <code>developedBy</code> , <code>github</code> , <code>developmentStart</code> , <code>currentVersion</code> , <code>updateHistory[]</code>
Markdown	optional eine Kopfzeile als HTML-Kommentar — nicht erforderlich, da Markdown direkt gerendert wird

Wichtig: `_meta` wird von der **Laufzeit-Logik nicht ausgewertet** — die Seite nutzt weiterhin Felder wie `files`, `title`, `slug`, `id`, `date`, `image`, `detail.contentFile` usw. Du kannst `_meta` aus einer Vorlage kopieren und nur **Datum / Autor / Beschreibung in `updateHistory`** anpassen, ohne die Funktion der Website zu beeinträchtigen.

Beispiel `_meta` (verkürzt):

```
{
  "_meta": {
    "project": "KI Netzwerk Schweinfurt",
    "pageName": "Event · ComfyUI Workshop",
    "pathInProject": "data/event/events/event-002-comfyui-workshop.json",
    "developedBy": "Oliver Braun",
    "github": "https://github.com/olli-br",
    "developmentStart": "2026-04",
    "currentVersion": "0.1.0",
    "updateHistory": [
      { "date": "2026-05-02", "author": "[Oliver Braun]", "changes":
"Project initialization" },
      { "date": "2026-05-08", "author": "[Oliver Braun]", "changes":
"Markdown-based content handling" }
    ]
  }
}
```

Neues Event anlegen

1. Eine bestehende Datei aus `data/event/events/` kopieren (z. B. `event-002-comfyUI-workshop.json`) **oder** die leere Vorlage `data/demo-files/event-entry.empty.json` als Startpunkt nutzen.
2. Neuen Dateinamen vergeben — nur `a-z`, `0-9`, Bindestrich, Endung `.json` (z. B. `event-003-mein-thema.json`).
3. JSON anpassen: `id`, `date` (ISO-Zeit), `image`, `link` (Anmeldung), `title`, `location`, `geo`, `detail.description`, `detail.contentFile` ... siehe [Schema: Event](#).
4. Zwei Markdown-Dateien anlegen für Detailseite (DE + EN):
 - `data/event/events/markdown/<name>.de.md`
 - `data/event/events/markdown/<name>.en.md`
5. Dateinamen in `data/event/index.json` unter `"files"` **ergänzen** (Reihenfolge = Anzeige-Reihenfolge in der Liste).
6. Optional: `_meta.updateHistory` mit Datum/Autor/Änderung ergänzen.

Sichtbar: Startseite (Kalender-Sektion „Nächste Events“, Countdown im Hero), `events.html` (Liste & Detail mit Karte), DE/EN über die Sprachumschaltung.

Neuen Blogbeitrag anlegen

1. `data/blog/posts/post-erstes-treffen.json` kopieren oder `data/demo-files/post-entry.empty.json` als Vorlage nehmen.
2. Neuen Dateinamen vergeben (z. B. `post-mein-artikel.json`).
3. JSON anpassen: `id`, `slug` (URL-Parameter — `blog.html?post=<slug>`), `date`, `image`, `title`, `teaser`, optional `readTime`, `detail.contentFile`. Details siehe [Schema: Blogbeitrag](#).
4. Zwei Markdown-Dateien für den Volltext (DE + EN):
 - `data/blog/posts/markdown/<name>.de.md`
 - `data/blog/posts/markdown/<name>.en.md`
5. Dateinamen in `data/blog/index.json` unter `"files"` **ergänzen**.
6. Falls der Beitrag ein Recap zu einem Event ist: Im zugehörigen Event-JSON `recapLink` auf `./blog.html?post=<slug>` setzen — die Event-Karte verlinkt dann zum Recap.

Sichtbar: Startseite (Sektion „Aktuelle Beiträge“), `blog.html` (Liste & Detail), DE/EN.

Markdown-Detailseiten

- Speicherort: `data/<bereich>/<eintrag>/markdown/<name>.<lang>.md`
- Sprachen: jeweils `.de.md` und `.en.md` parallel pflegen.
- In der JSON-Datei wird der **Pfad** unter `detail.contentFile.de` und `detail.contentFile.en` referenziert:

```
"detail": {
  "contentFile": {
    "de": "./data/event/events/markdown/event-001-kickoff.de.md",
    "en": "./data/event/events/markdown/event-001-kickoff.en.md"
  }
}
```

- Erlaubt sind alle gängigen Markdown-Elemente: Überschriften, Listen, **fett**, *kursiv*, **Code**, Codeblöcke, Bilder, Links, Tabellen, Zitate.
- Das Rendering erfolgt clientseitig über die `marked`-Bibliothek (CDN-Import on-demand). Ohne CDN-Verfügbarkeit greift ein einfacher Fallback-Renderer.

Seiten- & UI-Texte

Datei	Inhalt
<code>data/home/home-page.json</code>	Hero (Label, Titel, Text, Buttons), Hero-Panel, Kalender-Eyebrow, Recap-Eyebrow, Sponsoren (Liste mit Logo & Link), Mission (4 Features), FAQ-Eyebrows, CTA-Block
<code>data/about/about-page.json</code>	Eyebrow, Titel, Subtitle, Intro, Treffenbeschreibung (<code>meetingTitle</code> , <code>meetingLead</code> , <code>meetingBody</code> mit <code>\n\n</code> für Absätze), Teilnehmer-Chips, Partner-Block (Text, Chips, Logos), Initiatoren mit Bild & Bio

Datei	Inhalt
<code>data/event/events-page.json</code>	Page-Titel, Subtitle, Filter-Label, Filter-Placeholder, Detail-Labels (Zurück, Past/Next-Badge, Recap, Buttons), Map-Texte (Treffpunkt, Google Maps öffnen, Apple Maps öffnen, Kartenhinweis, Frame-Titel)
<code>data/blog/blog-page.json</code>	Page-Titel, Subtitle, Liste („Weiterlesen“, Lesezeit-Suffix, Recap-Badge), Detail (Zurück)
<code>data/global/global.json</code>	Navigation, Footer, FAQ-Inhalte, Theme-Labels, Common-Buttons, Countdown-Einheiten

Konvention: jede sichtbare Zeichenkette als Objekt `{ "de": "...", "en": "..." }`. Längere Texte mit Absätzen verwenden `\n\n`.

Schema: Event

Feld	Typ	Pflicht	Hinweis
<code>id</code>	Zahl oder String	ja	eindeutig
<code>date</code>	String (ISO)	ja	z. B. <code>2026-06-24T18:00:00</code>
<code>image</code>	String	ja	Pfad relativ zum Projektroot, z. B. <code>./assets/images/event-2.png</code>
<code>link</code>	String	ja	Anmelde-/Eveeno-/externe Seite
<code>recapLink</code>	String	nein	Link zu vergangenem Recap (eigene Blog-URL <code>./blog.html?post=<slug></code> oder externer Link)
<code>title</code>	<code>{ de, en }</code>	ja	
<code>location.venue</code>	<code>{ de, en }</code>	ja	Veranstaltungsort
<code>location.street</code>	String	ja	
<code>location.postalCode</code>	String	ja	
<code>location.city</code>	String	ja	
<code>geo.lat</code>	Zahl	ja	für Karten-Einbettung & Map-Links
<code>geo.lng</code>	Zahl	ja	siehe oben
<code>detail.description</code>	<code>{ de, en }</code>	ja	Kurztext (Karten-Teaser, Detail-Lead)

Feld	Typ	Pflicht	Hinweis
<code>detail.contentFile</code>	<code>{ de, en }</code>	ja	Pfade zu Markdown für die Detailseite

Status-Logik: „Vergangen“ und „Nächstes Event“ werden anhand des aktuellen Datums automatisch gesetzt; das nächstgelegene zukünftige Event erhält im Hero einen Countdown.

Schema: Blogbeitrag

Feld	Typ	Pflicht	Hinweis
<code>id</code>	String	ja	eindeutig
<code>slug</code>	String	ja	URL-Parameter <code>?post=</code>
<code>date</code>	String (ISO)	ja	Sortierung & Anzeige
<code>image</code>	String	ja	Cover-Bild
<code>title</code>	<code>{ de, en }</code>	ja	
<code>teaser</code>	<code>{ de, en }</code>	ja	Vorschau auf Karte und Listenseite
<code>readTime</code>	<code>{ de, en }</code>	empfohlen	z. B. <code>2 Min Lesezeit / 2 min read</code>
<code>detail.contentFile</code>	<code>{ de, en }</code>	ja	Pfade zu den Markdown-Volltexten

Lesezeit-Fallback: Fehlt `readTime` oder ist er leer, schätzt die Seite die Dauer aus Teaser + Markdown-Volltext und nutzt das Suffix aus `blog-page.json` → `list.readTime` (DE/EN).

Schema: About-Seite (Initiatoren & Partner)

`data/about/about-page.json` enthält neben den Texten zwei Spezial-Strukturen:

```
"participantChips": [
  { "de": "Führungskräfte & Teamleitungen", "en": "Leaders & team leads" }
],
"partnerChips": [
  { "de": "StudyFAB by netSWerk e.V.", "en": "StudyFAB by netSWerk e.V." }
],
"partnerLogos": [
  {
    "name": "StudyFAB",
    "src": "./assets/images/StudyFAB_Logo.jpeg",
    "href": "https://schweinfurt-fabulous.de/schweinfurt-fabulous/studyfab"
  }
],
"initiators": [
  {
```

```

    "name": "Morteza Djebeli Sinaki",
    "role": { "de": "...", "en": "..." },
    "bio": { "de": "Absatz 1\n\nAbsatz 2", "en": "..." },
    "image": "./assets/images/aboutme_1200x900.png",
    "linkedin": "https://www.linkedin.com/in/morteza-djebeli-sinaki"
  }
]

```

- **participantChips** / **partnerChips** werden als kleine Pills gerendert.
- **partnerLogos** zeigt klickbare Logo-Links unter dem Partner-Block.
- **initiators** rendert eine Karte pro Person (Bild + Bio + LinkedIn). Mehrere Einträge möglich.

Schema: Home-Seite (Sponsoren)

In **data/home/home-page.json**:

```

"sponsors": [
  {
    "name": "StudyFAB",
    "logo": "./assets/images/StudyFAB_Logo.jpeg",
    "link": "https://schweinfurt-fabulous.de/schweinfurt-fabulous/studyfab"
  }
]

```

Wird auf der Startseite im Sponsoren-Streifen angezeigt — beliebige Anzahl Einträge möglich.

Bilder & Medien

Damit alle Karten (Events, Blog, Home) **konsistent** dargestellt werden und die Bildhöhe **proportional zur Breite skaliert**, gilt ein einheitlicher Standard:

Seitenverhältnis	16 : 10 (für alle Karten- und Listenbilder)
Empfohlene Auflösung	1600 × 1000 px (oder ein Vielfaches davon)
Mindestauflösung	1200 × 750 px
Format	.jpg / .png für Fotos, .svg für Grafiken/Logos
Dateigröße (Foto)	< 350 KB (per Tool wie Squoosh / TinyPNG komprimieren)
object-fit	contain – das gesamte Motiv ist immer sichtbar, ggf. mit dezentem Rand

Ablage: alle Karten- und Beitragsbilder unter **assets/images/** (Pfad in JSON: **"image":** **"./assets/images/<datei>"**).

Technische Umsetzung: Höhen sind in `assets/css/styles.css` über `aspect-ratio: 16 / 10` definiert (`.card-image`, `.event-card__media`, `.blog-card .thumb-wrap`). Es gibt **keine festen height-Werte** mehr — Bilder skalieren von Mobile bis Ultra-Wide.

Hinweis: Wenn ein Bild abweichend formatiert ist (z. B. Hochformat), wird es per `object-fit: contain` mit Hintergrund zentriert angezeigt — vermeide das nach Möglichkeit, weil rechts/links sichtbarer Leerraum entsteht.

Sponsor- und Partner-Logos: möglichst quadratisch oder breitformatig auf transparentem/weißem Hintergrund; Höhe < 200 px reicht, da die Anzeige automatisch skaliert.

Lokaler KI-Assistent (WebLLM)

Die Website bindet einen **schwebenden Chat** ein (Partial `components/ai-chat.html`), der auf **allen Seiten** nach dem Laden von `assets/js/main.js` aktiv wird. Technisch basiert die KI-Antwort auf **WebLLM** (Paket `@mlc-ai/web-llm`): Das Sprachmodell wird im Browser geladen und mit **WebGPU** ausgeführt — **ohne** dass Nutzerfragen oder Antworten an einen eigenen Server des Projekts gesendet werden. (Der Browser lädt Modell-Gewichte und die WebLLM-Bibliothek von den konfigurierten CDNs.)

Ablauf in Kurzform

1. `data/global/ai-chat.json` wird mit den übrigen globalen Daten geladen: Labels (DE/EN), Liste der Modell-IDs (`models`), Standardwerte für Temperatur, Token-Länge, Kontextgröße usw. (`defaultSettings`), sowie `webLlmModuleUrls` — mehrere URLs, die nacheinander versucht werden, falls ein CDN blockiert oder ausfällt.
2. Beim ersten Senden einer Nachricht importiert der Code dynamisch das WebLLM-Modul (`importWebLlmModule` in `assets/js/main.js`), erzeugt eine **MLCEngine** mit dem gewählten Modell und dem Cache-Backend (**IndexedDB** oder **Cache API**, wählbar in den Chat-Einstellungen).
3. **Website-Kontext:** Vor der Antwort werden aus den bereits geladenen Seitendaten ein kompakter Text aufgebaut (`buildAiAssistantContext`): Startseite (Hero, Mission-Features, Sponsoren), FAQ-Auszug, die **nächsten sechs** Events (Sortierung wie in der Liste) und **vier** Blog-Teaser inkl. Slug — alles auf die Zeichenzahl `contextChars` (Standard in JSON, per UI anpassbar) begrenzt. Dieser Text steckt im **System-Prompt**; das Modell wird angewiesen, nur diesen Kontext zu nutzen und nichts zu erfinden.
4. **Antwort:** Es wird mit Streaming gearbeitet (bei Fehler einmaliger Versuch ohne Stream). **Abbrechen** unterbricht die Generierung (`interruptGenerate`).

Fallback ohne funktionierendes WebLLM

Wenn WebGPU fehlt, der Modul-Import scheitert oder das Modell nicht geladen werden kann, greift optional der **lokale Fallback** (Schalter „Lokalen Fallback nutzen“, `useFallback` in `defaultSettings` / `localStorage`):

- Es wird **kein** großes Sprachmodell ausgeführt, sondern eine **reine JavaScript-„Kontextsuche“** über dieselben Daten, die auch für den Chat-Kontext genutzt werden.
- `buildIntentFallbackAnswer` erkennt grob die **Absicht** anhand deutscher/englischer **Schlüsselwörter** in der Nutzerfrage (z. B. Event/Treffen/Wann → nächstes Event; Mitmachen/Kontakt

→ E-Mail-Hinweis; Themen/KI → kurzer Netzwerk-Fokustext mit optionalem Bezug zum nächsten Event; wer/was/Netzwerk → allgemeiner Kurzüberblick).

- Passt keine Intent-Route, durchsucht **buildLocalContextFallbackAnswer** eine **Kandidatenliste**: Hero- und Missionstext der Startseite, danach jedes Event und jeder Blogeintrag mit Titel sowie zusammengefasstem Fließtext (für Events z. B. Beschreibung, Adresse, Datum, Link; für Posts Teaser und Datum). Die Frage wird **normalisiert** (Kleinbuchstaben, Umlaute entflohchen, Sonderzeichen reduziert) und mit **scoreSearchCandidate** bewertet: Wörter mit mindestens **drei Buchstaben** aus der Frage werden im Text gezählt; die **drei besten Treffer** werden als Antwortvorschläge ausgegeben, sonst eine **Standardzusammenfassung** (Hero, nächstes Event, neuester Blog, Kontakt).

Das ist keine semantische Vektorsuche, sondern ein **leichtgewichtiger Scanner** über strukturierte Website-Inhalte — absichtlich ohne Backend und ohne Embedding-API.

Nutzer-Einstellungen & Pflege

Thema	Ort / Verhalten
Persistenz	localStorage -Schlüssel kins-ai-chat-settings (Modell, Schieberegler, Checkboxes, Akzentfarbe)
Texte / Modelle / CDNs	data/global/ai-chat.json — _meta wie bei anderen JSONs; Änderungen an labels, models, webLlmModuleUrls, defaultSettings
Zuverlässigkeit	Aktuellen Chrome-, Edge- oder Safari mit WebGPU über HTTPS oder localhost verwenden (siehe Hinweistexte in labels); bei Cache-Problemen IndexedDB als Cache-Speicher wählen

Dateien: **components/ai-chat.html** (Markup), Styles in **assets/css/styles.css** (Klassen **ai-chat***), gesamte Logik in **assets/js/main.js** (Funktionen **initAiChatAssistant, ensureAiAssistantEngine, buildAiAssistantContext, buildLocalContextFallbackAnswer, ...**).

Konventionen

- **Keine doppelten id** (Events) bzw. **id / slug** (Blog).
- **Dateinamen:** ausschließlich **a–z, 0–9**, Bindestrich; Endung **.json** bzw. **.md**.
- **Sprache:** überall **de** und **en** parallel pflegen — leere englische Texte führen sonst beim Sprachwechsel zu leeren Stellen.
- **_meta & Dateiköpfe:** bei inhaltlich größeren Änderungen **updateHistory** (JSON) bzw. die Update-Zeile im Kommentar (HTML/CSS/JS) ergänzen — einheitlich mit Datum (**YYYY-MM-DD**), Autor in eckigen Klammern und Kurzbeschreibung.
- **Pfade in JSON:** immer relativ, beginnend mit **./** (z. B. **./assets/images/...**, **./data/blog/posts/markdown/...**).
- **Markdown-Pfade:** auch hier **./** voranstellen; Klein-/Großschreibung im Pfad muss mit der tatsächlichen Datei übereinstimmen (manche Hoster und macOS sind hier streng bzw. nicht).
- **Zeitangaben:** ISO-8601 ohne Zeitzone (**YYYY-MM-DDTHH:MM:SS**); Anzeige & Countdown nutzen die lokale Zeitzone des Besuchers.

Deployment

Die Seite ist eine **reine Static-Site** und benötigt keinen Server. Geeignet sind u. a.:

- **GitHub Pages** — einfach den Repo-Branch (z. B. `main`) als Pages-Quelle wählen; Root des Repos = Webroot.
- **Netlify / Vercel / Cloudflare Pages** — als statische Seite ohne Build-Schritt deployen (kein `package.json` erforderlich).
- **Eigener statischer Webspaces** — alle Dateien einfach hochladen; wichtig ist nur, dass die relative Verzeichnisstruktur erhalten bleibt (`./components/`, `./data/`, `./assets/` müssen vom Root aus erreichbar sein, damit die `fetch`-Aufrufe funktionieren).

Vor dem Deployment empfiehlt sich:

1. JSON-Validität prüfen (z. B. `python3 -m json.tool data/event/index.json` und `data/global/ai-chat.json`).
2. Lokal mit `python3 -m http.server` testen, sowohl in DE als auch EN.
3. Bilder optimieren (Größe, Format).
4. `index.json` (Events & Blog) auf die korrekte Reihenfolge prüfen.
5. **KI-Chat**: Auf der Ziel-URL mit **HTTPS** testen (WebGPU ist dort zuverlässiger als bei beliebigen `http://`-Hosts); prüfen, ob die CDNs unter `webLlmModuleUrls` erreichbar sind.

Projekt eigenständig entwickelt von Oliver Braun. Code gerne zum Lernen nutzen — bitte die Attribution in den Dateiköpfen und in `_meta` respektieren.